

Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу

«Операционные системы»

Группа: М8О-207БВ-25

Студент: Расторгуев А.Г.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 03.10.26

Москва, 2026

Постановка задачи

Вариант 2.

Родительский процесс создает дочерний процесс. Первой строчкой пользователь в консоль родительского процесса пишет имя файла, которое будет передано при создании дочернего процесса. Родительский и дочерний процесс должны быть представлены разными программами. Родительский процесс передает команды пользователя через `pipe1`, который связан с стандартным входным потоком дочернего процесса. Дочерний процесс при необходимости передает данные в родительский процесс через `pipe2`. Результаты своей работы дочерний процесс пишет в созданный им файл. Допускается просто открыть файл и писать туда, не перенаправляя стандартный поток вывода.

Пользователь вводит команды вида: «число число число `<endline>`». Далее эти числа передаются от родительского процесса в дочерний. Дочерний процесс считает их сумму и выводит её в файл. Числа имеют тип `float`. Количество чисел может быть произвольным.

Общий метод и алгоритм решения

`pipe()` — создание каналов для обмена данными между родительским и дочерним процессами.

`fork()` — создание дочернего процесса.

`dup2()` — перенаправление стандартного ввода дочернего процесса в канал.

`read()` — чтение данных из консоли и каналов.

`write()` — запись данных в каналы и файл.

`open()` — создание и открытие файла для записи результата.

`close()` — закрытие ненужных файловых дескрипторов.

`exec1()` — запуск дочерней программы с передачей ей аргументов.

`waitpid()` — ожидание завершения дочернего процесса.

Программа состоит из двух отдельных процессов: родительского и дочернего, которые взаимодействуют через два канала `pipe`. Родительский процесс сначала считывает с клавиатуры имя файла, в который должен быть записан результат. Затем с помощью `pipe()` создаёт два канала и через `fork()` создаёт дочерний процесс.

После создания процесса родитель запускает дочернюю программу с помощью `exec1()`, передавая ей имя файла и дескриптор второго канала. При этом первый канал используется для передачи команд от родителя к ребёнку, а второй — для передачи сообщений от ребёнка к родителю. Родитель считывает введённые пользователем строки с числами и передаёт их дочернему процессу через первый канал. Дочерний процесс получает эти данные через стандартный ввод, который был перенаправлен на канал с помощью `dup2()`.

Дочерний процесс разбирает полученную строку, преобразует содержащиеся в ней числа в числовой формат и вычисляет их сумму. Полученный результат записывается в указанный ранее файл. После обработки строки дочерний процесс отправляет родителю сообщение через второй канал. Родитель получает это сообщение и продолжает ожидать следующую строку. Пустая строка используется для завершения работы.

Код программы

parent.c

```
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>
#include <string.h>

int main()
{
    char filename[256];
    char command[4096];

    int pipe1[2]; // parent -> child
    int pipe2[2]; // child -> parent

    int pos = 0;
    char c;

    while (pos < 255)
    {
        if (read(STDIN_FILENO, &c, 1) <= 0)
            return 1;

        if (c == '\n')
            break;

        filename[pos++] = c;
    }

    filename[pos] = '\0';

    if (pipe(pipe1) == -1 || pipe(pipe2) == -1)
        return 1;

    pid_t pid = fork();

    if (pid == -1)
        return 1;

    if (pid == 0) // child process
    {
        close(pipe1[1]);
        close(pipe2[0]);

        if (dup2(pipe1[0], STDIN_FILENO) == -1)
            _exit(1);
```

```

close(pipe1[0]);

char pipe2_fd[20];

int len = 0;
int fd = pipe2[1];

char temp[20];

if (fd == 0)
{
    temp[len++] = '0';
}
else
{
    while (fd > 0)
    {
        temp[len++] = '0' + fd % 10;
        fd /= 10;
    }
}

for (int i = 0; i < len; i++)
    pipe2_fd[i] = temp[len - i - 1];

pipe2_fd[len] = '\0';

execl("./child", "child", filename, pipe2_fd, NULL);

_exit(1);
}
else // parent process
{
    close(pipe1[0]);
    close(pipe2[1]);

    while (1)
    {
        pos = 0;

        while (pos < 4095)
        {
            if (read(STDIN_FILENO, &c, 1) <= 0)
            {
                close(pipe1[1]);
                close(pipe2[0]);
                waitpid(pid, NULL, 0);
                return 0;
            }

```

```

        }

        command[pos++] = c;

        if (c == '\n')
            break;
    }

    int sent = 0;

    while (sent < pos)
    {
        int n = write(pipe1[1], command + sent, pos - sent);

        if (n <= 0)
        {
            close(pipe1[1]);
            close(pipe2[0]);
            waitpid(pid, NULL, 0);
            return 0;
        }
        sent += n;
    }

    char response[256];

    int n = read(pipe2[0], response, sizeof(response) - 1);

    if (n > 0)
    {
        response[n] = '\0';

        write(STDOUT_FILENO, response, n);
    }

    if (pos == 1 && command[0] == '\n')
        break;
}

close(pipe1[1]);
close(pipe2[0]);
waitpid(pid, NULL, 0);

return 0;
}
}

```

child.c

```
#include <unistd.h>
#include <fcntl.h>
#include <stdlib.h>
#include <string.h>

int main(int argc, char *argv[])
{
    if (argc < 3)
        return 1;

    // argv[1] – file name.
    // argv[2] – descriptor pipe2.

    char *filename = argv[1];

    int pipe2_fd = atoi(argv[2]);

    int file = open(
        filename,
        O_WRONLY | O_CREAT | O_TRUNC,
        0644);

    if (file == -1)
        return 1;

    char buffer[4096];

    while (1)
    {
        int size = read(STDIN_FILENO, buffer, sizeof(buffer));

        if (size <= 0)
            break;

        int end = -1;

        for (int i = 0; i < size; i++)
        {
            if (buffer[i] == '\n')
            {
                end = i;
                break;
            }
        }

        if (end == 0)
            break;
    }
}
```

```

char line[4096];

int line_size = end;

if (line_size >= sizeof(line))
    line_size = sizeof(line) - 1;

for (int i = 0; i < line_size; i++)
    line[i] = buffer[i];

line[line_size] = '\0';

char *ptr = line;

float sum = 0.0f;

while (*ptr != '\0')
{
    char *next;

    float number = strtod(ptr, &next);

    if (next != ptr)
    {
        sum += number;
        ptr = next;
    }
    else
    {
        ptr++;
    }
}

int negative = 0;

if (sum < 0)
{
    negative = 1;
    sum = -sum;
}

int integer_part = (int)sum;
int fraction = (int)((sum - integer_part) * 1000000.0f);

char result[128];
int r = 0;

if (negative)

```



```

        result[r++] = '-';

char digits[32];
int d = 0;

if (integer_part == 0)
{
    digits[d++] = '0';
}
else
{
    while (integer_part > 0)
    {
        digits[d++] = '0' + integer_part % 10;
        integer_part /= 10;
    }
}

for (int i = d - 1; i >= 0; i--)
    result[r++] = digits[i];

result[r++] = '.';

int divisor = 100000;

for (int i = 0; i < 6; i++)
{
    result[r++] = '0' + fraction / divisor % 10;
    divisor /= 10;

    if (divisor == 0)
        divisor = 1;
}

result[r++] = '\n';

write(file, result, r);

char message[] = "Сумма записана в файл\n";

write(pipe2_fd, message, strlen(message));
}

close(file);
close(pipe2_fd);

return 0;
}

```


Протокол работы программы

```
child.c M test1 U X
lab1 > test1
1 16.500000
2 0.100030
3 100501.023449
4 1368.000000
5 140.000000
6 0.999800
7 8665.991000
8

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● rastral@Asus-vivobook:~/OS-labs-2026-27/lab1$ ./parent
test1
1.1 2.2 3.3 4.4 5.5
Сумма записана в файл
0.00001 0.00002 0.1
Сумма записана в файл
100500 0.12345 0.9
Сумма записана в файл
123 456 789
Сумма записана в файл
123 -23 45 -5
Сумма записана в файл
1 -0.0002
Сумма записана в файл
-100 -0.009 -1234 10000
Сумма записана в файл
○ rastral@Asus-vivobook:~/OS-labs-2026-27/lab1$
```

Вывод

В ходе лабораторной работы была реализована организация взаимодействий между родительским и дочерним процессом. Была написана программа, которая складывает числа типа float и записывает результат в текстовый файл.